

ML & Backend-Notes

Folder structure:

```
edge-vision-project/
├── backend/           # The FastAPI Microservice
│   ├── app/
│   │   ├── api/      # routes.py (The "Receptionist" - URL endpoints)
│   │   ├── core/     # config.py, logger.py, exceptions.py (The rulebook)
│   │   ├── schemas/  # pydantic_models.py (The "Security Guard" validating JSON)
│   │   ├── services/ # ml_engine.py (The "Engine Room" - PyTorch inference)
│   │   └── main.py   # Ignites the FastAPI server
│   ├── models/       # edge_digit_vision_final.pth (The Model Vault)
│   ├── Dockerfile    # Container instructions for the backend
│   └── requirements.txt # Backend dependencies
├── frontend/         # The Streamlit Microservice
├── ui.py            # The interactive canvas
├── Dockerfile        # Container instructions for the frontend
└── requirements.txt  # Frontend dependencies
```

1. **schemas/** (The Security Guard)

When Streamlit sends the image over the internet to FastAPI, it arrives as a massive string of text (Base64 encoding).

Before PyTorch even sees it, it must pass through your **schemas**. We use a library called **Pydantic** here. It checks the incoming JSON payload and says: *"Is this actually an image string? Is it empty? Is it malicious?"* If it fails the check, FastAPI rejects it immediately.

2. **api/** (The Receptionist)

Also known as the "Routers." This folder contains the actual URL endpoints (e.g., **POST** **/predict/digit**).

The receptionist's only job is to take the verified JSON from the security guard, hand it to the back room, and wait for the result. *There is zero machine learning code written here.*

3. **services/** (The Engine Room)

This is where your PyTorch logic lives. The **api** hands the image string to a file here (e.g., **ml_service.py**).

This file decodes the string, resizes it to 28×28, runs **model(img)**, extracts the predicted digit (e.g., "7"), and hands the number "7" back to the receptionist.

4. **main.py** (The Factory Manager)

This is the ignition switch. It is usually only 15 lines of code. It just starts the FastAPI server, connects the `api` routers, and opens the factory for business.

5. `models/` (The Vault)

This is just a folder holding your `edge_digit_vision_final.pth` file. When the server boots up, `main.py` goes to the vault, loads the model into RAM once, and keeps it hot and ready.

How Streamlit and FastAPI Talk

Because we are decoupling them, Streamlit and FastAPI will run as two completely independent programs. They don't share Python memory.

They communicate using **REST API calls** (JSON over HTTP).

1. User draws a '4' on Streamlit.
2. Streamlit packages the image into a JSON envelope: `{"image_data": "base64_string_here..."}`.
3. Streamlit fires an HTTP POST request across the internet to your FastAPI URL.
4. FastAPI's factory processes it and replies with JSON: `{"prediction": 4, "confidence": 98.5}`.
5. Streamlit reads the reply and prints it on the screen.

This architecture means if you wanted to build an Android app tomorrow, you wouldn't have to rewrite any PyTorch code. The Android app would just ping the exact same FastAPI URL.

A. Why did we abandon the Training Pipeline in the API?

You remember building a massive `src/` folder in Flask that handled data ingestion, transformation, and model training. It is completely correct that doing it that way allows the system to train on new data.

So why don't we do that in FastAPI? It isn't a difference between Flask and FastAPI. It is a difference between old-school web dev and modern **MLOps** (Machine Learning Operations).

In modern systems, we strictly separate the **Factory (Training)** from the **Storefront (Inference/API)** to ensure reliability.

- **The Problem with the Old Way:** Training a neural network takes immense compute power (GPUs) and time. If your API server is busy retraining a model on new data, and a user sends a request to predict a digit, the server will freeze, and the user gets a "504 Gateway Timeout."
- **The Modern Professional Way:** We build deployable, scalable systems by isolating the compute.
 - The **Training Pipeline** lives somewhere else entirely (like GitHub Actions, Airflow, or a dedicated cloud server). It runs automatically at midnight, trains on new data, creates a new `.pth` file, and silently drops it into your `models/` vault.

- The **FastAPI Server** only handles Inference. It is lightning-fast, uses very little RAM, and its only job is to serve users. If it detects a new `.pth` file in the vault, it simply hot-swaps it into memory without ever dropping a user request.

B. The "Two Digits" Edge Case (e.g., drawing "58")

Your intuition is spot on. If someone notoriously draws a "58" side-by-side on the canvas, **the model will confidently fail.**

Your PyTorch CNN does not have a concept of "two numbers" or "I don't know." It is mathematically forced to look at the edges and curves of the 28×28 image and assign it to one of the 10 buckets (0-9) it was trained on. It will likely crush the "58" together, look at the curves, and spit out something like an "8" with 60% confidence.

How professionals handle this:

1. **The Quick Fix (Product Level):** We add strict instructions on the Streamlit UI: *"Please draw a SINGLE digit in the center of the canvas."*
2. **The Engineering Fix (System Level):** If you were deploying this to read real-world checks or addresses, you would add an **Object Detection** step (like YOLO or OpenCV Contours) *before* the classifier. The system would scan the image, draw a bounding box around the "5", draw a bounding box around the "8", split them into two separate 28×28 images, run your PyTorch model twice, and stitch the answers together.

For this specific web portfolio piece, we will rely on the UI constraint, but knowing *why* it fails is exactly.

C. Why do we need ml_engine? The entire Architecture again while we have already trained in .ipynb file and exported .pth file..... ?

It feels redundant, right? You already built it, trained it, and exported it. Why are we writing the exact same `MNISTVisionModel` class again in VS Code?

Here is the exact reason: **A `.pth` file is not a program. It is just a giant list of numbers.**

To understand why we need `ml_engine.py`, let's look at the "Brain and Memories" analogy.

The Brain vs. The Memories

When you used `torch.save(model_0.state_dict(), "edge_digit_vision_final.pth")` in Google Colab, you used the `state_dict()` command.

- **The Python Code (`MNISTVisionModel` class):** This is the physical structure of the **Brain**. It defines exactly how many lobes there are (the Convolutional layers), how they connect, and how data

flows through them.

- **The `.pth` file (`state_dict`):** These are the **Memories** (the weights and biases) that the brain learned during training.

If you try to load "memories" into your backend, FastAPI will look at the `.pth` file and say: *"Okay, I have 30,000 numbers here... but what do I pour them into? I don't have a brain structure to hold them!"*

Because PyTorch's `state_dict` does not save the Python code, **you must recreate the exact empty brain structure in your API before you can pour the memories back into it.**

D. Why not save the whole model?

You might be wondering: *"Why didn't we just save the whole brain AND the memories in one file?"*

PyTorch *does* have a command for that (`torch.save(model)`), but **professionals almost never use it**. Saving the entire model ties the file directly to the exact folder structure and exact version of PyTorch used in Colab. If you try to load it on a different server (like Hugging Face), it will often break and throw a "Directory Not Found" error.

Saving *just* the weights (`state_dict`) is mathematically pure. It makes your `.pth` file incredibly lightweight and guarantees it will run on any server on Earth, as long as you provide the blueprint (`ml_engine.py`) for it to live in.

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

<https://hand-digit-recognition-system-from-scratch-cphfpx9zzvzf8s3zp3.streamlit.app/>